C# 프로그래밍

학습목표

- C#에 대해서 이해할 수 있다.
- C#을 이용해 프로그램을 제작할 수 있다.

개요

Unity 엔진의 스크립트 언어는 C#이다. 따라서 Unity 엔진을 가지고 여러 가지를 구현하려면 C#에 대해서 잘 이해하고 있어야 한다. 이번 강의에서는 Unity C#을 다루기 위한 기본적인 부분에 대해서 배우게 된다.

C#의 동작

C#과 C++의 차이점 중 하나가 C#은 네이티브 언어(Native Language)*가 아니라는 것이다. C#으로 작성된 코드는 **IL**(Intermediate Language)로 컴파일 되며, IL 코드와 리소스는 함께 어셈블리(assembly)에 저장된다. 그리고 C# 프로그램이 실행되면 이 어셈블리가 **.NET**에 로드된다. NET은 **CLR**(Common Language Runtime)**으로 고수준 언어와 운영체제 사이에 위치해 있어 고수준 언어를 각 운영체제가 이해할 수 있는 언어로 **JIT**(Just-In-Time) 컴파일*** 한다.

* 네이티브 언어 : 운영체제가 직접 대응되는 기계어로 변환시킬 수 있는 언어. 반대는 관리 언어(Managed Language)다.
** CLR : CLI(Common Lanugage Infrastructure) 구현체
*** JIT 컴파일 : 프로그램을 실제 실행하는 시점에 기계어로 변환하는 것

메모리 관리

C#과 C++의 차이점 중 두 번째는 메모리를 자동으로 관리해준다는 것이다. 다시 말해 메모리 누수를 신경쓰지 않아도 된다. 이것이 가능한 이유는 가비지 수집기(Garbage Collector) 때문이다. 가바지 수집기는 매니지드 힙(Managed Heap)을 통해 메모리를 관리하는 데 하나씩 살펴보자.

메모리 할당

매니지드 힙에는 객체가 할당될 주소를 가리키는 포인터가 있어, 이를 통해 메모리를 할당하게 된다. 각 객체들은 주소 공간이 허락하는 한 인접한 곳에 위치하게 된다. 매니지드 힙을 이용하면 스택에서 메모리를 할당하는 속도만큼 빠르게 할당할 수 있고, 접근도 빠르게 할 수 있다.

메모리 해제

가비지 수집기는 자동으로 가장 적합한 시간을 결정해 수집을 수행한다. 수집이 수행될 땐 애플리케이션에서 더 이상 사용되지 않는 객체에 대한 메모리를 해제하게 된다. 이렇게 더 이상 사용되지 않는 객체를 가비지라고 한다.

가비지는 애플리케이션의 루트를 이용해 결정되는데, 이러한 루트에는 정적 필드, 스레드 스택의 지역 변수, 매개 변수, CPU 레지스터가 있다. 이러한 루트를 통해 접근할 수 있는 객체를 전부 포함한 그래프를 만들어, 이 그래프에 없는 객체를 가비지로 간주한다.

매니지드 힙을 검사해 가비지를 찾으면 접근할 수 있는 객체를 압축해 가비지의 공간을 덮어버린다. 그 후 포인터를 접근할 수 있는 객체 다음에 지정하게 된다. 큰 객체의 메모리(85KB 이상)는 **LOH**(Large Object Heap)라는 별도 힙에 할당해 관리하는데, 이 메모리는 압축되지 않는다.

수집을 수행한다고 하여 무조건적으로 메모리 압축이 일어나는 것은 아니고 가비지의 개수가 엄청나게 발견되는 경우에만 메모리 압축이 일어나게 된다.

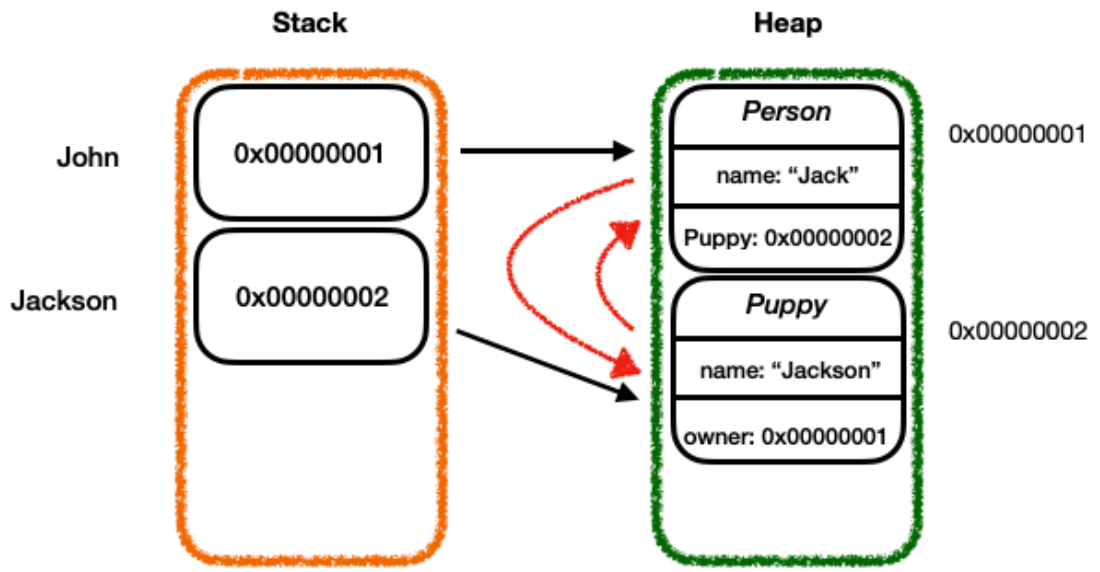
세대

가비지 수집기는 세대(generation)를 나눠서 관리한다. 0세대, 1세대 및 2세대의 총 3개 세대가 있다. 세대를 나눈 이유는 (1) 더 빠르게 메모리를 압축할 수 있고, (2) 새로운 객체일수록 수명이 짧기 때문이고, (3) 새로운 객체일수록 서로 연결되는 경향이 있어 애플리케이션에서 거의 동시에 접근되기 때문이다.

새로운 객체는 0세대에 저장된다. 0세대가 가득 차면 수집이 수행되는데, 연결할 수 있는 객체의 메모리를 압축한 후 세대를 승격시킨다. 만약 0세대에서 수집을 수행했음에도 불구하고 새로운 객체를 만들기 위한 메모리 공간이 충분하지 않다면 세대 2, 1, 0의 순서로 수집을 수행한다. 이 경우에도 세대 승격은 일어난다.

약한 참조

가비지 수집기는 사용되지 않는 메모리를 수집한다. 다시 말하자면 참조 횟수를 이용하여 메모리 수집 여부를 결정한다. 하지만 객체 간에 아래와 같은 참조 관계를 가지게 되면 어떻게 될까?



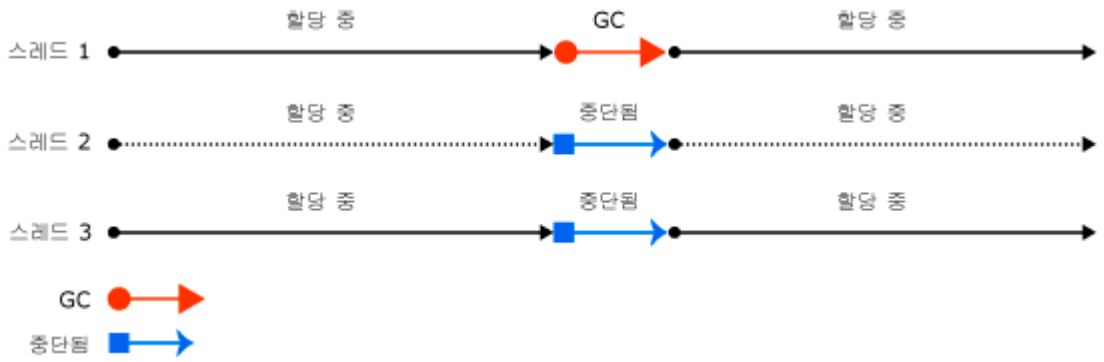
스택에서 John과 Jackson 모두 사라지더라도, 인스턴스 간에 서로를 가리키고 있어서 참조 횟수가 줄어들지 않기 때문에 메모리 누수가 발생하게 된다. 이런 상황을 순환 참조(Circular Reference)라 한다.

이러한 순환 참조를 방지하려면 약한 참조(Weak Reference)*를 사용해야 한다. 약한 참조는 참조 횟수에 관여하지 않아 인스턴스 간에 서로 참조하고 있더라도 가비지 수집이 정상적으로 작동되게 된다.

* 반대로 참조 횟수를 늘리는 참조를 강한 참조(Strong Reference)라 한다.

효율적인 코드 작성하기

가비지 수집은 성능과 직결된다. 가비지 수집 자체에 컴퓨팅 자원을 쓰는 이유도 있지만, 가비지 수집 중에는 다른 스레드가 일시 중단되기 때문이다.



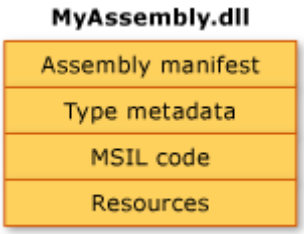
따라서 위의 동작 방식에 근거해서 코드를 효율적으로 작성해야 한다.

- 객체를 너무 많이 할당하면 안된다.
  - 빈번한 할당은 빈번한 수집을 초래할 수 있다.
- 너무 큰 객체 할당을 피해야 한다.
  - LOH에서는 메모리 압축이 실행되지 않아 내부 단편화가 발생할 수 있다.
- 복잡한 참조 관계를 만들지 말자.
  - 가독성도 문제지만 수집 후 메모리 주소 관리를 어렵게 한다. 특히나 오래된 세대에서 새로운 세대에 대한 메모리를 참조하게 될 때 수집을 방지하기 위해 쓰기 장벽(Write Barrier)을 만드는데 이는 많은 성능을 필요로 한다.
- 루트를 너무 많이 만들지 말자.
  - 루트 목록이 작을 수록 수집을 수행하는 횟수가 줄어든다.

어셈블리

C# 프로그램의 실체는 어셈블리이다. 어셈블리는 서로 함께 사용되어 논리적 기능 단위를 형성하도록 빌드되는 타입 및 리소스의 컬렉션을 의미한다. 어셈블리는 실행 파일(.exe) 또는 동적 연결 라이브러리(.dll)의 형태를 가지며, .NET 기반 애플리케이션에 대한 배포, 버전 제어, 재사용, 활성화 범위 및 보안 권한의 기본 단위를 형성한다.

어셈블리는 일반적으로 아래의 네 가지 요소로 구성된다.

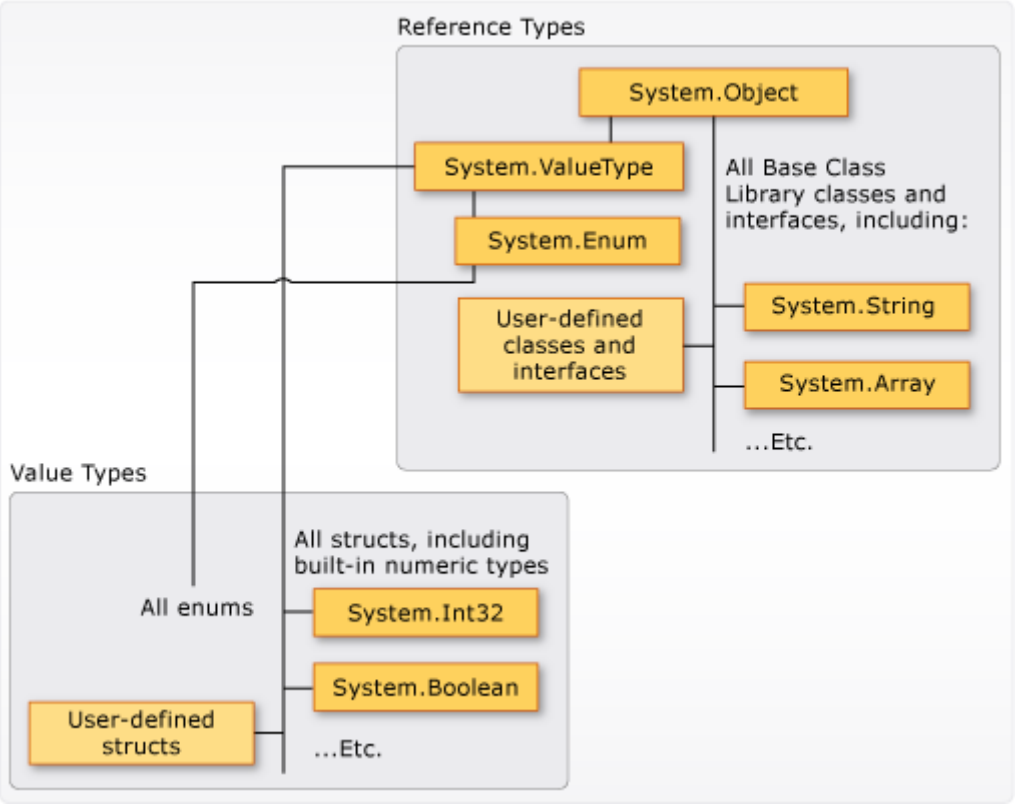


어셈블리 매니페스트는 어셈블리의 메타데이터를 표시한다. 구체적으로는 아래와 같은 기능을 수행한다.

- 어셈블리를 구성하고 있는 파일을 열거한다.
- 어셈블리의 타입 및 리소스에 대한 참조가 해당 선언 및 구현이 포함된 파일에 매핑되는 방법을 관리한다.
- 어셈블리가 사용하고 있는 다른 어셈블리를 열거한다.
- 어셈블리 사용자와 구현체 간의 간접 참조 수준을 제공한다.
- 어셈블리가 자체 설명되도록 렌더링한다.

공용 타입 시스템

.NET에서는 여러 .NET 언어를 지원하기 위해 CTS(Common Type System)을 지원한다. .NET의 모든 형식은 값 타입(Value Type) 혹은 참조 타입(Reference Type)으로 구분되며 모든 타입은 기본 타입인 System.Object에서 파생된다. C#에서는 아래와 같은 계층을 가지고 있다.



값 타입

C#에서 값 타입은 구조체 / System.Enum / 그 외에 기본 제공 타입으로 구성된다. 값 타입은 아래와 같은 특징을 갖는다.

- 구조체를 제외한 모든 타입은 System.ValueType에서 파생된다.
- 스택 메모리에 직접 값이 포함된다. 다시 말해 복사가 일어난다.
- 상속이 불가능(sealed)하다.
- 구조체 멤버 중에 참조 타입이 있다면 참조가 복사된다.

참조 타입

C#에서 참조 타입은 클래스 / 대리자 / 배열 / 인터페이스이 있다. 참조 타입은 아래와 같은 특징이 있다.

- 매니지드 힙에 메모리가 할당된다. 다시 말해 가비지 수집의 영향을 받는다.
- 참조 타입의 변수는 인스턴스의 위치에 대한 참조를 가진다.
- null을 할당할 수 있다.

박싱과 언박싱

박싱(boxing)은 값 타입을 object 타입 또는 값 타입에서 구현된 임의의 인터페이스 타입으로 변환하는 프로세스며, 언박싱(unboxing)은 박싱된 인스턴스에서 값 타입을 추출하는 프로세스다. 박싱이 이뤄질 땐 값이 힙으로 복사되며, 언박싱이 이뤄질 땐 힙에서 스택으로 값이 복사된다. 또, 박싱은 암시적으로 이뤄지며, 언박싱은 명시적으로 이뤄진다.

속성

속성(Property)은 필드(Field)의 확장된 버전이라고 생각하면 된다. 속성을 이용하면 좀 더 적은 코드로 필드와 관련된 메소드를 작성할 수 있다. 아래는 속성 사용 예시이다.

```
// C++
class Person
{
public:
    std::string GetFirstName() const { return _firstName;}
    std::string GetSecondName() const { return _secondName; }
    std::string GetFullName() const { return _firstName +
_secondName; }

    void SetFirstName(const std::string& firstName) { _firstName =
firstName; }
    void SetSecondName(const std::string& secondName) { _secondName =
secondName; }
private:
    std::string _firstName;
    std::string _secondName;
}

// C#
class Person
{
    public string FirstName { get; set; }
    public string SecondName { get; set; }
    public string FullName
    {
        get { return $"{FirstName} + {SecondName}"; }
    }
}
```

인덱서

인덱서(Indexer)는 C++에서 []연산자를 만드는 것과 같으며, 속성 문법을 따른다.

```
// C++
class MyVector
{
public:
    int&    operator[](size_t index) { return _container[index]; }
    const int& operator[](size_t index) const { return
_container[index]; }
private:
    int*    _container;
}

// C#
class MyVector
{
    private int[] _container;

    public int this[int index]
    {
        get { return _container[index]; }
        set { _container[index] = value; }
    }
}
```

식 본문 멤버

식 본문(Expression Body) 정의를 이용하면 좀 더 간결한 코드를 작성할 수 있다. C# 6에서는 메서드와 읽기 전용 속성에서만 사용 가능하다.

```
public class Person
{
   public Person(string firstName, string lastName)
   {
      fname = firstName;
      lname = lastName;
   }

   private string fname;
   private string lname;
```

```
    public override string ToString() => $"{fname} {lname}".Trim();
    public void DisplayName() => Console.WriteLine(ToString());
}
```

동등성, 동일성

동등성(Equality)은 값이 같은 걸 의미하고, 동일성(Identity)은 같은 객체인 걸 의미한다. 동등성은 Equals()나 ==을 통해 비교할 수 있으며, 동일성은 ReferenceEquals()를 통해 알 수 있다.

확장 메서드

확장 메서드(Extension Method)는 기존 타입을 수정하지 않고도 기존 타입에 메서드를 추가할 수 있다. 사용 예제는 여기를 참고한다. 아래는 Unity 엔진에서 문자열을 사용할 때 좀 더 성능이 좋은 커스터마이징 버전의 EndsWith() / StartsWith() 다.

```
public static class Extensions
{
    public static bool CustomEndsWith(this string a, string b)
    {
        int ap = a.Length - 1;
        int bp = b.Length - 1;

        while (ap >= 0 && bp >= 0 && a [ap] == b [bp])
        {
            ap--;
            bp--;
        }

        return (bp < 0);
    }

    public static bool CustomStartsWith(this string a, string b)
    {
        int aLen = a.Length;
        int bLen = b.Length;

        int ap = 0; int bp = 0;

        while (ap < aLen && bp < bLen && a [ap] == b [bp])
        {
            ap++;
            bp++;
```

```
        }

        return (bp == bLen);
    }
}
```

제네릭

C#에서는 일반화 프로그래밍(Generic Programming)*을 지원하기 위해 제네릭이라는 기능이 있다. 간단한 사용 방법은 여기를 참고하자.

*일반화 프로그래밍 : 타입을 파라미터로 전달 받아 해당 타입에 대한 코드를 필요할 때 생성하는 프로그래밍 방식

런타임의 제네릭

C++과 다르게 제네릭은 런타임에 특수화된 타입이 만들어진다. 특수화된 타입이 만들어진 후에는 해당 타입을 재사용한다. 주의할 점은 참조 타입의 제네릭 타입에 대해서는 단일의 타입만 만들어진다는 것이다.

제약 조건

타입 파라미터로 전달될 수 있는 타입 인자에 조건을 달 수 있다. 제약 조건을 달면 좀 더 타입에 안전한 코드를 작성할 수 있다. 제약 조건에 대해선 여기를 참고하라. 이와 비슷한 기능으로 C++에서는 concepts라는 것이 있다.

컬렉션

C++과 비슷하게 C#에서도 다양한 자료구조를 System.Collections에서 제공하고 있다. 하지만 모든 타입이 System.Object로 고정되어 있어 오류가 일어나기 쉽고, 값 타입에 대해서는 박싱 이슈가 있기 때문에 System.Collections.Generic에 있는 컬렉션을 사용하는 것이 좋다. 컬렉션 이슈에 대해서는 여기를 참고하라. C++과 매칭되는 컬렉션은 아래와 같다.

- unordered_map : Dictionary
- unordered_set : HashSet
- list : LinkedList
- vector : List
- priority_queue : PriorityQueue
- queue : Queue
- map : SortedDictionary
- set : SortedSet
- stack : Stack

템플릿과의 차이점

C#의 제네릭은 C++의 템플릿과 동일한 수준의 유연성을 제공하고 있진 않다. 또한 특수화도 템플릿은 마치 매크로와 같이 전처리가 일어나며, 제네릭은 런타임에 해당 타입으로 대체된다. 아래에서 실제 코드 수준에서의 차이점을 알아보자

- 논타입 템플릿 매개 변수를 허용하지 않는다.

```
// in c++
template <size_t N>
class A
{

};
// in c#
class A<int N> // 컴파일 오류
{

}
```

- 명시적 특수화를 지원하지 않는다.

```
// in c++
template <typename T>
class A
{

};

// 가능
template <>
class A<int>
{

};
// in c#
class A<int> // 컴파일 오류
{

}
```

- 부분 특수화를 지원하지 않는다.

```
// in c++
template <typename T, typename U>
class A
{

};
```

```
template <typename T, int>
class A
{

};
// in c#
class A<T, U>
{

}

class A<T, int> // 컴파일 오류
{

}
```

- 타입 파라미터를 제네릭 타입에 대한 부모 클래스로 사용할 수 없다.

```
// in c++
template <typename T>
class A : public T
{

};
// in c#
class A<T> : T // 컴파일 오류
{

}
```

- 타입 파라미터에 기본 타입을 지정할 수 없다.

```
// in c++
template <typename T = int>
class A
{

};

A<> a; // A<int>로 만들어짐.

// in c#
class A<T = int> // 컴파일 오류
```

```
{

}
```

- 제네릭 타입 파라미터는 제네릭이 될 수 없다.

```
// in c++
template <typename T, template <typename> typename C>
class A
{
    C<T> a;
};
// in c#
class A<T, C>
{
    C<T> a;
}
```

- 제약 조건으로 특정 조건을 충족하는 타입만 타입 인자로 받을 수 있도록 할 수 있다.

```
// in c++
template <typename T>
class A
{
    public void foo()
    {
        a.boo(); // T 타입에 boo()가 없을 수 있음
    }

    T a;
};

// in c#
interface IBoo
{
    void Boo();
}
class A<T> where T : IBoo
{
    T a;

    public void Foo()
```

```
    {
        // 제약 조건으로 IBoo를 구현한 타입에 대해서만
        // 특수화 가능
        a.Boo();
    }
}
```

반복자 메소드

C++에서는 STL 컨테이너를 순회할 때 반복자를 사용한다. 마찬가지로 C#에서도 반복자가 있다. C#에서는 순회할 수 있는 타입인 IEnumerable과 IEnumerable 타입을 순회할 수 있는 IEnumerator가 있다.

C#에서는 이런 타입을 반환하는 특수한 메소드를 만들 수 있다. 이를 반복자 메소드라고 한다. 여기서는 yield문이라는 특수한 문을 사용할 수 있다.

반복자 메소드의 실행은 MoveNext()를 통해 이뤄지며, yield문을 만날 때까지 실행된다. 만약 첫 MoveNext() 실행 후 함수의 끝까지 다다르지 않았다면, 다음 MoveNext()에서는 이전에 끝낸 yield문의 다음 구문부터 실행된다. yield return문에서 반환된 값은 Current에서 접근할 수 있다.

대리자 및 이벤트

대리자

대리자(delegate)는 함수 포인터와 비슷한 참조 타입으로 기능은 System.Delegate와 System.MulticastDelegate로부터 받게 된다. 아래는 사용 예시이다.

```
class TimeManager
{
    public static TimeManager Instance { get; }

    // delegate 객체
    // 만드는 방법은 여기를 참고하라
    public SecondElapsedDelegate OnSecondElapsed = logTime;

    // delegate 만들기
    // (Access Specifier) delegate [Return Type] [Identifier]
[Parameter List];
    private delegate void SecondElapsedDelegate();

    private float _totalTime = 0f;
    private float _lastTime = 0f;

    void Update()
```

```
    {
        _totalTime += Time.time;

        if (_totalTime - _lastTime >= 1f)
        {
            _lastTime = _totalTime;

            // delegate에 연결된 메소드 전부 호출
            // 혹은 OnSecondElapsed.Invoke();
            OnSecondElapsed();
        }
    }

    private void logTime()
    {
        Console.WriteLine($"1 sec elapsed.");
    }
}

class Plant
{
    public Plant()
    {
        // -= 및 += 연산자를 이용해 delegate에 메소드 삭제 및 추가 가능
        TimeManager.Instance.OnSecondElapsed -= grow;
        TimeManager.Instance.OnSecondElapsed += grow;
    }
    private void grow()
    {
        Console.WriteLine("자람");
    }
}
```

.NET에는 미리 만들어진 대리자가 있는데 Action과 Func가 그것이다. 둘은 반환형의 유무 차이 밖에 없다.

C++ 함수 포인터와의 차이점

함수 포인터와 비슷한 컨셉을 갖고 있지만 몇 가지 측면에서 다른 점이 있다.

- 함수 포인터는 어떤 메서드의 주소를 가르키지만, 대리자는 내부적으로 객체의 주소와 메서드 주소를 함께 갖고 있어, 정적 메서드나 인스턴스 메서드 등을 가리지 않고 다룰 수 있다.
- 함수 포인터는 멤버 함수를 가리킬 시, 한 타입에 대해서만 사용할 수 있지만 대리자는 여러 타입의 메소드를 참조할 수 있다.

- 함수 포인터는 단일의 메소드만 참조할 수 있지만 대리자는 여러 개의 메소드를 참조할 수 있다.
- 함수 포인터는 타입 안전성이 없지만, 대리자는 타입 안전성을 갖고 있다.

이벤트
이벤트는 옵저버 패턴을 좀 더 사용하기 쉽게 구현된 기능이다. 클래스 내부의 변경 사항을 구독자에게 알리는 데 사용한다. 아래는 사용 예제다.

```
class TimeManager
{
    public static TimeManager Instance { get; }

    // 이벤트 인수는 해당 이벤트가 발생될 때, 구독자에게 바뀐 상태를 전달하기
위한 데이터다.
    // 보통 EventArgs에서 파생한다.
    public TimeElapsedEventArgs : EventArgs
    {
        public float LastTime { get; set; }
        public float TotalTime { get; set; }
        public float ElapsedTime { get { return TotalTime - LastTime;
} }

        public TimeElapsedEventArgs(float lastTime, float totalTime)
        {
            LastTime = lastTime;
            TotalTime = totalTime;
        }
    }

    // Event 객체
    // [Access Specifier] event EventHandler (Identifier)
    public event EventHandler<TimeElapsedEventArgs> OnSecondElapsed;

    private float _totalTime = 0f;
    private float _lastTime = 0f;

    void Update()
    {
        _totalTime += Time.time;

        if (_totalTime - _lastTime >= 1f)
        {
            _lastTime = _totalTime;
```

```
            // 구독자에게 이벤트 발생했음을 알림
            // 구독자가 없으면 null일 수 있기에 null 체크를 한다.
            OnSecondElapsed?.Invoke(this, new
TimeElapsedEventArgs(_lastTime, _totalTime));
        }
    }
}

class Plant
{
    public Plant()
    {
        // -= 및 += 연산자를 이용해 이벤트에 구독 혹은 해제 가능
        // 무명 메소드를 추가할 수도 있지만, 취소가 필요한 경우라면 안 쓰는
것이 좋다.
        TimeManager.Instance.OnSecondElapsed -= grow;
        TimeManager.Instance.OnSecondElapsed += grow;
    }

    private void grow(object sender, TimeManager.TimeElapsedEventArgs
args)
    {
        Console.WriteLine($"이전 시간은 {args._lastTime}");
        Console.WriteLine("자람");
    }
}
```

리플렉션

리플렉션(Reflection)은 동적으로 어셈블리나 모듈, 타입에 대한 정보를 제공하는 시스템이다. 프로그래밍 레벨에서 어떤 타입의 멤버에 관한 정보를 가져오거나, 메소드를 가져오거나 하는 등의 동작을 할 수 있다. 리플렉션 시스템의 핵심은 System.Reflection과 Type이다. 아래 예시를 보자.

```
using System;
using System.Reflection;

class Program
{

    class A
```

```
    {
        public int temp = 10;
        private int temp2 = 20;
        static internal string temp3;
    }


    static void Main()
    {
        A a = new A();

        // Type 객체는 GetType()을 호출하거나 typeof 연산자를 통해 가져올 수
있다.
        // a.GetType()과 typeof(A)는 동일하다.
        Type type = a.GetType();

        // Type에는 여러가지 메소드가 있는데, 아래는 그 중 하나다.
        // GetRuntimeFields()는 클래스 A의 모든 필드에 관한 정보를 가져온다.
        var fields = type.GetRuntimeFields();
        foreach (FieldInfo field in fields)
        {
            Console.WriteLine($"Name : {field.Name} / Static? :
{field.IsStatic} / Type : {field.FieldType} / Value :
{field.GetValue(a)}");
        }
    }


}
```

특성

특성(Attribute)은 메타데이터나 선언적 정보를 코드에 연결할 수 있게 해주는 특수한 클래스다. 특성은 리플렉션 시스템을 통해 접근하여 동적으로 사용할 수 있다. 주로 사용하는 특성으로는 ObsoleteAttribute, ConditionalAttribute, AttributeUsageAttribute 등이 있다. 다양한 특성을 살펴보려면 여기를 참고하자. 예시는 아래와 같다.

```
using System;
using System.Reflection;

// 특성의 사용은 []안에 특성 이름을 적으면 된다.
```

```
// 특성을 적용할 대상을 지정할 수 있으며, 자세한 것은 여기를 참고하자.
[assembly: AssemblyDescription("특성 예제 프로그램")]

class Program
{
    // 사용자 정의 특성을 만드려면 Attribute를 상속 받으면 된다.
    // 특성 이름은 접미사로 Attribute를 붙여준다.
    class CustomAttribute : Attribute
    {

    }

    // AttributeUsageAttribute를 이용해 특성을 적용할 대상을 지정할 수 있다.
    // 기본은 AttributeTargets.All이다.
    [AttributeUsage(AttributeTargets.Class)]
    class Custom2Attribute : Attribute
    {

    }

    // 특성은 여러 개를 지정할 수 있다.
    // 이외에도 [Custom][Custom2]처럼 적을 수도 있다.
    [Custom, Custom2]
    class A
    {
        [Custom]
        public void foo()
        {
            Console.WriteLine("Hello, Attribute");
        }

        // Custom2는 클래스 레벨에서만 적용 가능하다.
        // 따라서 컴파일 오류가 난다.
        [Custom2]
        public void boo()
        {
            Console.WriteLine("Hello");
        }
    }

    class Custom3Attribute : Attribute
    {
```

```
        public readonly string Message;
        public string Version;

        // 특성에는 매개변수를 전달할 수 있다.
        public Custom3Attribute(string message)
        {
            Message = message;
        }
    }

    // 일반 public 멤버는 아래처럼 초기화 할 수 있다. 필수는 아니다.
    [Custom3("This is Test", Version="1.0.0")]
    class B
    {

    }


    static void Main()
    {
        B b = new B();

        // 특성은 GetCustomAttribute를 통해서 가져올 수 있다.
        // 특성 객체는 이때 만들어진다. 리플렉션 시스템을 통해서 접근하기
전까지는 절대 만들어지지 않는다.
        Custom3Attribute attribute =
b.GetType().GetCustomAttribute<Custom3Attribute>();

        // 만들어진 뒤에는 일반 클래스처럼 사용하면 된다.
        // Output : This is Test
        Console.WriteLine(attribute.Message);
    }
}
```

특성을 이용해서 유니온을 만들 수 있다. 자세한 것은 여기를 참고하자.

참고자료

- C# 문서 - 시작, 자습서, 참조.
- C# 스터디
- The Memory Management Reference